



Entornos de Desarrollo

*UT4- Optimización y
documentación*

Refactorización

Técnica, que consiste en realizar **pequeñas transformaciones** en el código de un programa, para **mejorar la estructura** sin que **cambie el comportamiento ni funcionalidad** del mismo.

Su objetivo es mejorar la estructura, la legibilidad o la eficiencia del código, cumpliendo la misma función

$$(x-1)(x-1)=$$

Factorización

$$x^2 + 1$$



Refactorización

Se podría entender como el **mantenimiento del código** para facilitar su comprensión, pero sin añadir ni eliminar funcionalidades.

Refactorizar código consiste en **crear un código más limpio**

Se refactoriza para:

- **Limpieza del código**, mejorando la consistencia y la claridad
- **Código duplicado**, **métodos** demasiado **largos**, **clases** muy **grandes** o con demasiados métodos (Bad Smells)
- **Elimina** el código "**muerto**" (Clases con pocas llamadas o que se usan muy poco) y se modulariza
- **Facilita** el futuro mantenimiento y **modificación** del código

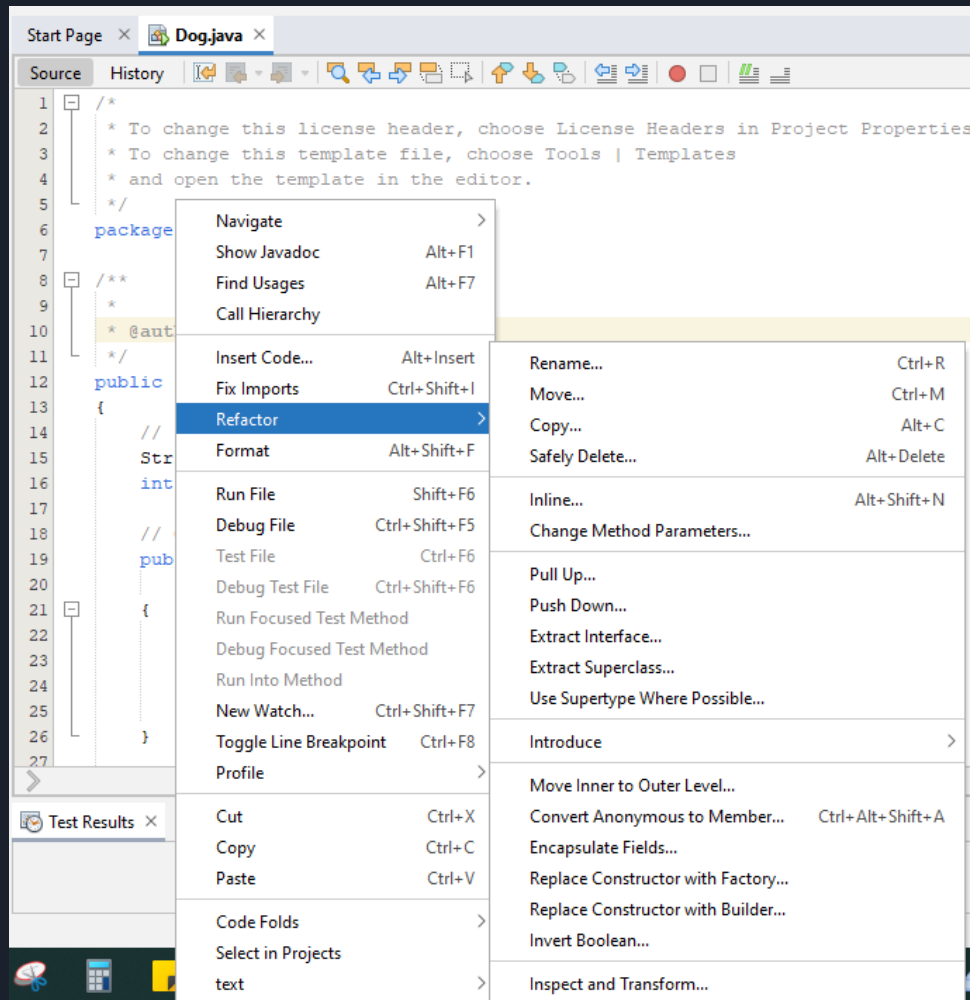
Refactorización

Cuando se habla de refactorizar, es muy importante conocer algunos **los aspectos del lenguaje**, por ejemplo en **Java**:

- Las Convenciones de escritura
- Ficheros
- Declaración de variables
- Nombres de los identificadores
- Magic Numbers
- Estructura del código
- Bad Smells
- Buenas prácticas:
 - Manejo de Strings
 - Visibilidad de atributos
 - Constantes, variables...

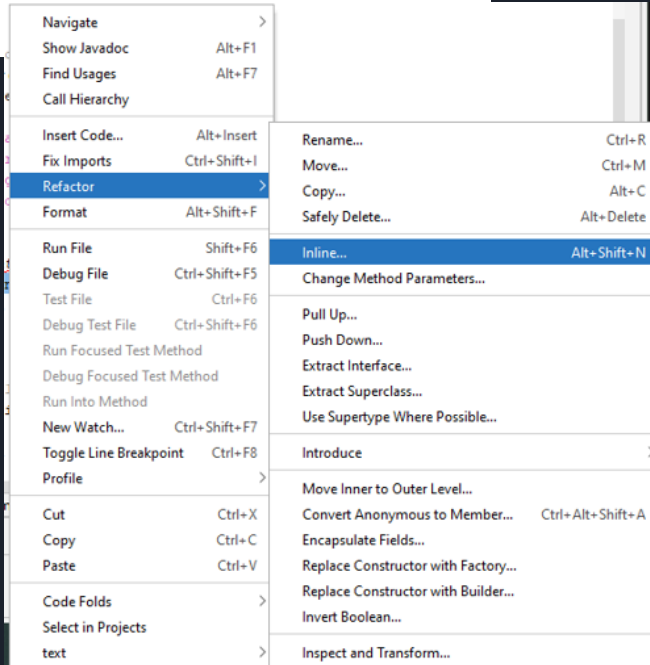


Refactorización: Manual o NetBeans



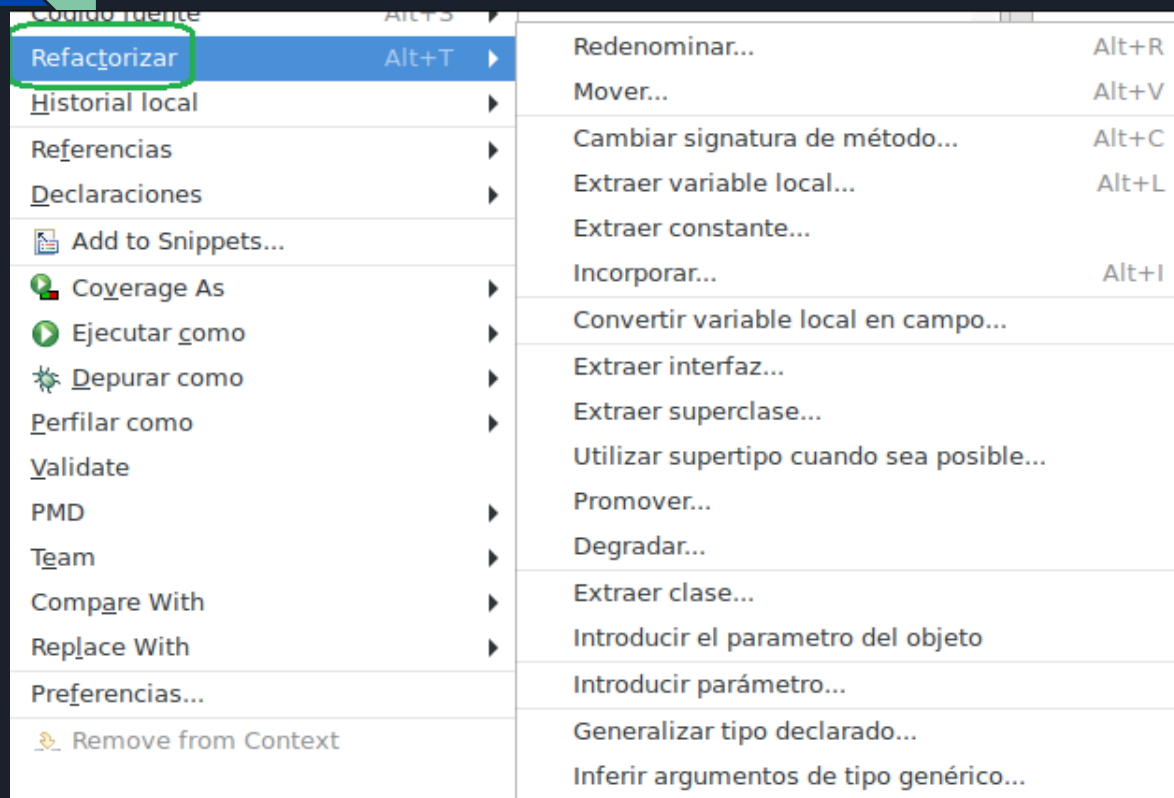
Refactorización: NetBeans

```
public Punto simétrico() {  
    Punto nuevoP = new Punto (this.x * -1, this.y);  
    return nuevoP;  
}
```



```
public Punto simétrico() {  
    return new Punto (this.x * -1, this.y);  
}
```

Refactorización: Manual o Eclipse



**Ver ejemplos en los
contenidos del
CAMPUS**

Refactorización: Eclipse

```
public class clasecolor {  
    private static final String PRIMER_PARAMETRO = "Primer Parametro";  
  
    public static void main(String[] args) {  
        profesor tch = new profesor();  
        String color = tch.preguntacolor();  
        String sResp = "La respuesta recibida es:";  
        System.out.println(sResp + color);  
  
        cambSig(10,5);  
    }  
}
```



Convertir variable local en campo

Nombre de campo:

Modificador de acceso

☐ public ☐ protected ☐ package ☒ private

Inicializar en

☐ Declaración de campo
☒ Método actual
☐ Class constructors

☒ Declarar campo como 'static'
☐ Declarar campo como 'final'



```
public class clasecolor {  
    private static final String PRIMER_PARAMETRO = "Primer Parametro";  
    private static String color;  
  
    public static void main(String[] args) {  
        profesor tch = new profesor();  
        color = tch.preguntacolor();  
        String sResp = "La respuesta recibida es:";  
        System.out.println(sResp + color);  
  
        cambSig(10,5);  
    }  
}
```




Analizadores de código

Recordemos que hay **pruebas de código** estáticas y dinámicas en el desarrollo de software:

Dinámicas: Se comprueba si el código sigue un conjunto de reglas, además de ejecutar las pruebas unitarias

Estáticas: se hace el análisis estático de código, directamente sobre el código fuente, pero sin llegar a ejecutarlo

Existen **herramientas**, como los **analizadores de código**, cuya función es encontrar partes del código que puedan reducir el rendimiento, provocar errores en el software, tener una excesiva complejidad, complicar el flujo de datos o crear problemas de seguridad

Si esto se hace de manera **manual** se habla de **revisión de código**

Analizadores de código



Un ejemplo es [PMD](#), una herramienta para Java que basa su funcionamiento en detectar patrones, que son posibles errores en tiempo de ejecución, código que no se puede ejecutar nunca porque no se puede llegar a él, código que puede ser optimizado, expresiones lógicas que pueden ser simplificadas, malos usos del lenguaje, etc.

Otro analizador de código disponible en el mercado es [Sonargube](#), herramienta Open-Source de análisis de calidad del código disponible para gran cantidad de lenguajes de programación



Sistema de control de Versiones Git-GitHub

Un **Sistema de control de versiones** permite llevar un registro del estado por el que han pasado todos los archivos del proyecto:

Cuándo, quién y qué cambios se han realizado, además de volver a un estado anterior

- Compartir, validar y hacer cambios experimentales con el código

- Mantener varias copias (distribuidas en varios equipos y que se están actualizando a la vez)

- Actualizar el proyecto completo en un servidor

- Hacer comprobaciones (calidad) antes del despliegue de un proyecto

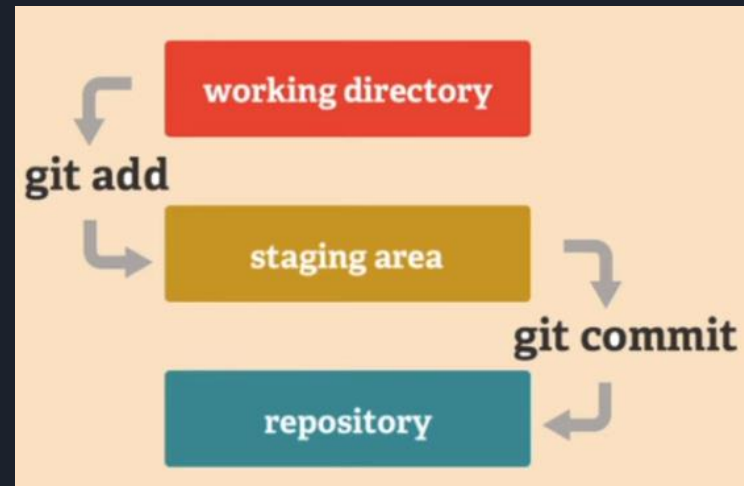


Git-GitHub

GIT es un sistema de control de versiones distribuido de código abierto utilizado para el control de cambios en el desarrollo de software. Permite a los desarrolladores trabajar en el mismo proyecto de forma colaborativa y mantener un historial completo de los cambios realizados en el código

GitHub Es una plataforma de desarrollo colaborativo para alojar proyectos (en la nube) utilizando el sistema de control de versiones Git. Además cuenta con una herramienta muy útil que es GitHub Pages donde podemos publicar nuestros proyectos estáticos (HTML, CSS y JS) gratis

Git-GitHub

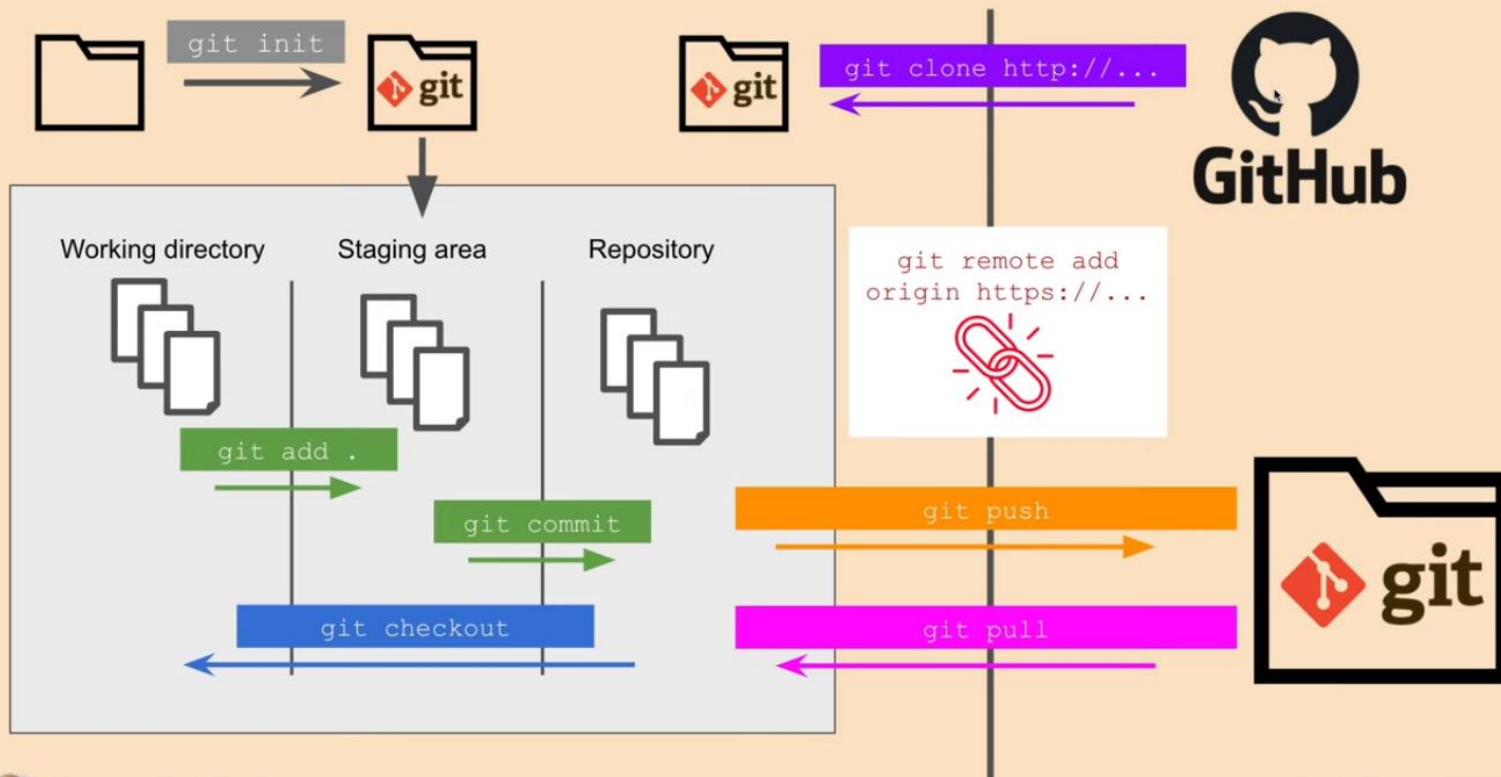


Working directory: área donde están todos los archivos en una carpeta en mi equipo. Sólo yo veo lo que hay, lo puedo editar y nadie controla

Staging area: cuando se añade (*add*) un archivo aquí significa que ya está en proceso de estar listo y Git comienza a controlar las modificaciones

Repository: una vez confirmo (*commit*) un archivo a esta área es que ya está listo y el resto puede verlo

Git-GitHub



Miguel Angel Alvarez
@midesweb

Control de versiones

Git-GitHub y los entornos de desarrollo



15/06/2021

Contenidos

1. **Introducción a Git**
2. Introducción a GitHub
3. Repositorios en NetBeans
4. Repositorios en IntelliJIDEA
5. Repositorios en phpStorm
6. Repositorios en Microsoft Visual Studio



Documentación con JavaDocs

Documentar con suficiente detalle un proyecto es muy importante para realizar **mantenimientos** futuros, conocer qué hacen las clases, métodos, interfaces, variables, etc. y cómo lo hacen

Esto va a servir a otr@s programador@s a **entender** de una forma fácil todo lo que se hizo en un proyecto y saber para qué sirve cada cosa

Documentación con JavaDocs

Javadoc es una utilidad para generar documentación de APIs y proyectos en formato HTML (partiendo del código fuente Java)

Javadoc es el estándar para documentar, de forma automática, clases de Java, por lo que se encuentra en la mayoría de los IDE específicos para Java (Eclipse, NetBeans...)

Documentación [JavaDocs](#)

